

Grado en Ingeniería Electrónica de Comunicaciones
Procesamiento de Señales · Curso 2026-2027

Apuntes

Tema 2. Muestreo y reconstrucción de señales

Jesús Chacón Sombría

Cómo usar estos apuntes

Estos apuntes desarrollan el contenido de las cuatro presentaciones del tema: las secciones 1–3 corresponden a la parte 1 (muestreo y aliasing), la sección 4 a la parte 2 (reconstrucción), la sección 5 a la parte 3 (cuantificación y codificación) y la sección 6 a la parte 4 (muestreo de señales discretas). Los ejemplos de código están en `teoría/T2/codigo/`, en Python y en Matlab, y se pueden ejecutar tal cual. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

Índice general

1 Del mundo analógico al digital 2
1.1 La cadena de procesado . . . 2

2 Muestreo periódico y aliasing 3
2.1 Frecuencia continua y frecuencia discreta . . . 3
2.2 Alias . . . 4

3 El teorema del muestreo 5
3.1 Modelo con un tren de impulsos . . . 6
3.2 Espectro de la señal muestreada . . . 6
3.3 Visión continua y visión discreta . . . 7
3.4 Filtro antialiasing . . . 8
3.5 Muestreo no ideal . . . 9

4 Reconstrucción 10
4.1 Reconstrucción ideal . . . 10
4.2 Por qué la reconstrucción ideal no es realizable . . . 11
4.3 La reconstrucción como interpolación . . . 12
4.4 Retenedor de orden cero . . . 12
4.5 Retenedor de orden uno . . . 13
4.6 Comparación y conversión D/A . . . 14

5 Cuantificación y codificación 15
5.1 Cuantificador uniforme . . . 15
5.2 Redondeo y truncamiento . . . 16
5.3 Ruido de cuantificación . . . 17
5.4 Codificación . . . 18
5.5 Parámetros de un conversor A/D . . . 19

6 Muestreo de señales discretas 19
6.1 Muestreo de una secuencia . . . 20

6.2 Diezmado 20
6.3 Expansión e interpolación 21
6.4 Cambio de frecuencia por un factor racional 22
6.5 Conversores con sobremuestreo 23

Resumen del tema **23**

Bibliografía **24**

1 Del mundo analógico al digital

Casi todas las señales que interesa procesar son analógicas: la presión acústica, la tensión de un electrodo, la temperatura o el campo electromagnético que recibe una antena. Durante buena parte del siglo XX se procesaron con circuitos analógicos: el predictor de mareas de Lord Kelvin (1873) sumaba sinusoides con poleas y engranajes, y desde los años cuarenta los amplificadores operacionales permitieron construir sumadores, integradores y filtros. Son rápidos, pero sensibles al ruido y a la deriva de los componentes, y poco flexibles: cambiar el procesado exige cambiar el circuito.

El procesado digital sustituye el circuito por un programa. Su despegue se apoya en tres hitos: el algoritmo FFT de Cooley y Tukey (1965), que hizo práctico calcular espectros (Gauss había usado una idea equivalente hacia 1805), el Intel 2920 (1979), un procesador de señal con conversores integrados, y el TMS32010 de Texas Instruments (1983), el primer DSP de éxito comercial, con arquitectura Harvard y una instrucción de multiplicar y acumular en un ciclo.

1.1 La cadena de procesado

Para procesar digitalmente una señal analógica se sigue la cadena de la figura 1. El filtro antialiasing limita el ancho de banda de la entrada; el conversor A/D la *muestrea* (toma valores en instantes discretos), la *cuantifica* (redondea cada valor a uno de un conjunto finito) y la *codifica* (asigna a cada valor una palabra binaria). El procesador opera sobre la secuencia de números y, si la salida tiene que volver a ser analógica, el conversor D/A y el filtro de suavizado la reconstruyen.



Figura 1. Cadena de procesado digital de una señal analógica.

La figura 2 muestra los tres pasos de la conversión A/D. Cada uno introduce un tipo de error distinto, y este tema estudia cuándo y cuánto:

- El **muestreo** no pierde información si la señal tiene banda limitada y se muestrea lo bastante deprisa (apartados 2 y 3). Si no, aparece el *aliasing*.
- La **reconstrucción** ideal recupera la señal exactamente, pero no es realizable; los retenedores la aproximan (apartado 4).
- La **cuantificación** siempre pierde información, pero el error se puede hacer tan pequeño como se quiera añadiendo bits (apartado 5).

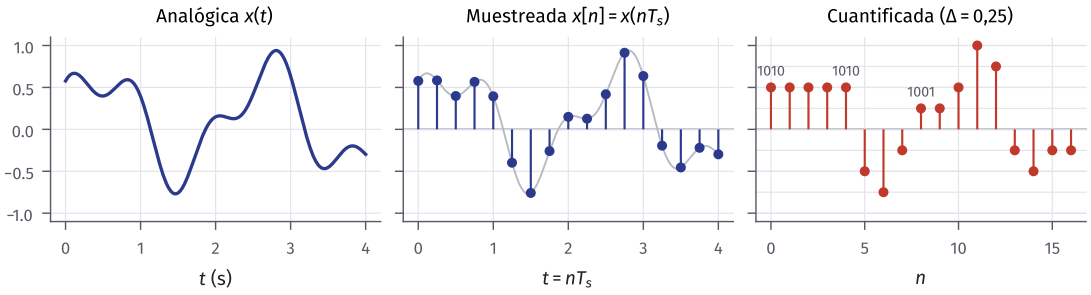


Figura 2. Señal analógica, señal muestreada (tiempo discreto, amplitud continua) y señal cuantificada (digital).

2 Muestreo periódico y aliasing

Definición 2.1: Muestreo periódico

Muestrear periódicamente una señal $x(t)$ con *periodo de muestreo* T_s es formar la secuencia

$$x[n] = x(nT_s), \qquad n \in \mathbb{Z}.$$

La *frecuencia de muestreo* es $f_s = 1/T_s$ en Hz, o $\omega_s = 2\pi f_s = 2\pi/T_s$ en rad/s.

La secuencia no guarda T_s: es una lista de números. Los mismos números representan una señal lenta muestreada despacio o una rápida muestreada deprisa, y solo al conocer T_s (o f_s) se puede volver a la escala de tiempo física (figura 3).

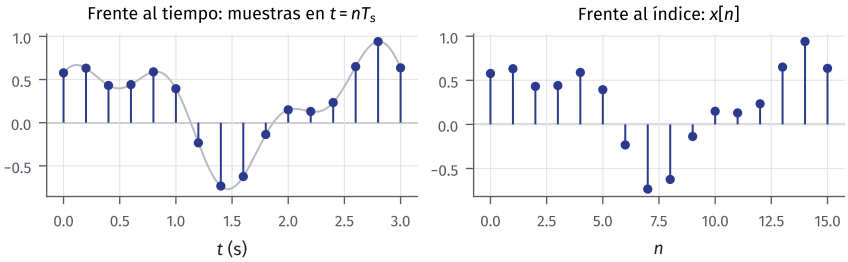


Figura 3. Muestreo periódico de una señal continua.

2.1 Frecuencia continua y frecuencia discreta

Si se muestrea una sinusoide $x(t) = A\cos(\omega_0 t + \phi)$, se obtiene

$$x[n] = A\cos(\omega_0 T_s n + \phi) = A\cos(\Omega_0 n + \phi), \qquad \Omega_0 = \omega_0 T_s = 2\pi\frac{f_0}{f_s}.$$

La frecuencia discreta Ω (rad/muestra) es la frecuencia continua ω (rad/s) *normalizada* por la frecuencia de muestreo. En ciclos por muestra, $F_0 = f_0/f_s$. Esta relación,

$$\boxed{\Omega = \omega T_s = \frac{2\pi\omega}{\omega_s}} \tag{1}$$

es la clave de todo el tema y la que permite traducir especificaciones en hercios a frecuencias discretas y viceversa.

Ejemplo 2.2: Una frecuencia en tres unidades

Un tono de $f_0 = 1\,\text{kHz}$ muestreado a $f_s = 8\,\text{kHz}$ tiene $\omega_0 = 2000\pi\,\text{rad/s}$, $F_0 = 1/8$ ciclos por muestra y $\Omega_0 = \pi/4\,\text{rad/muestra}$: cada periodo del tono ocupa ocho muestras. Si el mismo tono se muestrea a $f_s = 48\,\text{kHz}$, $\Omega_0 = \pi/24$.

2.2 Alias

En el Tema 1 vimos que las frecuencias discretas Ω_0 y $\Omega_0 + 2\pi k$ dan la misma secuencia, porque $e^{j2\pi kn} = 1$. Deshaciendo el cambio (1), todas las frecuencias continuas

$$\omega_k = \omega_0 + k\,\omega_s, \qquad k \in \mathbb{Z},$$

producen exactamente las mismas muestras. Como el coseno es par, también $-\omega_0 + k\omega_s$. Cada una de estas sinusoides es un *alias* de las demás: al muestrear se vuelven indistinguibles (figura 4).



Figura 4. Dos sinusoides de frecuencias distintas que dan las mismas muestras.

Definición 2.3: Alias fundamental

El *alias fundamental* de una sinusoide muestreada es el de menor frecuencia: el único con $|\Omega| \le \pi$ o, lo que es lo mismo, con $|\omega| \le \omega_s/2$. Se obtiene llevando Ω_0 al intervalo $[-\pi, \pi)$ con

$$\Omega_a = \big((\Omega_0 + \pi) \bmod 2\pi\big) - \pi.$$

Si la sinusoide original cumple $|\omega_0| < \omega_s/2$, ella misma es el alias fundamental y no hay ambigüedad. Si no, existe una sinusoide de frecuencia menor con las mismas muestras y cualquier reconstrucción que produzca señales de frecuencia menor que $\omega_s/2$ devolverá esa y no la original. Este fenómeno se llama *aliasing* (en castellano, a veces, solapamiento).

En hercios, una sinusoide de frecuencia f_0 aparece en la *frecuencia aparente*

$$f_a = |f_0 - k f_s| \in [0, f_s/2], \qquad k = \text{el entero más próximo a } f_0/f_s.$$

El eje de frecuencias «se pliega» en los múltiplos de $f_s/2$, que se llama *frecuencia de Nyquist* (figura 5). Es lo que hace que las ruedas de un coche parezcan girar despacio o al revés en un vídeo, o que un tono de 30 kHz grabado a 48 kHz sin filtrar suene a 18 kHz.

Ejemplo 2.4: Alias de una sinusoide y de una suma

Con $T_s = 2\,\text{s}$:

- $\sin\left(\frac{2\pi}{10}t\right)$ tiene $\Omega_0 = \frac{4\pi}{10} < \pi$: ya es su alias fundamental.

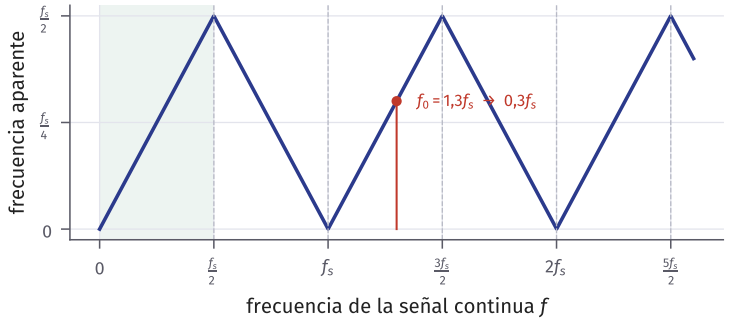


Figura 5. Frecuencia aparente en función de la frecuencia de la sinusoide muestreada.

- $\sin\left(\frac{2\pi}{10/6}t\right)$ tiene $\Omega_0 = \frac{24\pi}{10}$. Restando 2π, $\Omega_a = \frac{4\pi}{10}$, que corresponde a $\omega = \Omega_a/T_s = \frac{2\pi}{10}$ rad/s. Las dos señales dan las mismas muestras.
- $x(t) = 3\sin\left(\frac{2\pi}{5}t\right) + \sin\left(\frac{2\pi}{3}t\right)$: el primer término tiene $\Omega_1 = \frac{4\pi}{5} < \pi$ y se conserva; el segundo, $\Omega_2 = \frac{4\pi}{3}$, que se pliega a $\frac{4\pi}{3} - 2\pi = -\frac{2\pi}{3}$, es decir, $\omega = -\frac{\pi}{3}$ rad/s. Como el seno es impar, el alias fundamental es

$$3\sin\left(\frac{2\pi}{5}t\right) - \sin\left(\frac{\pi}{3}t\right).$$

En código

Cálculo del alias fundamental de la segunda sinusoide del ejemplo 2.4.

Python

```python
import numpy as np

def alias(w, Ts):
    """Frecuencia discreta en π[-, π)."""
    W = w * Ts
    return (W + np.pi) % (2*np.pi) - np.pi

Ts = 2.0                          # s
w1 = 2*np.pi/10                   # rad/s
w2 = 2*np.pi/(10/6)               # rad/s
print(alias(w1, Ts), alias(w2, Ts))  # π0.4
n = np.arange(20)
x1 = np.sin(w1 * Ts * n)
x2 = np.sin(w2 * Ts * n)
print(np.allclose(x1, x2))        # True
```

Matlab

```matlab
% Frecuencia discreta en π[-, π)
alias = @(w, Ts) mod(w*Ts + pi, 2*pi) - pi;

Ts = 2;                           % s
w1 = 2*pi/10;                     % rad/s
w2 = 2*pi/(10/6);                 % rad/s
disp([alias(w1, Ts), alias(w2, Ts)])  % π0.4
n = 0:19;
x1 = sin(w1 * Ts * n);
x2 = sin(w2 * Ts * n);
disp(max(abs(x1 - x2)) < 1e-9)  % 1
```

Ejercicios 1, 2 y 6. *Herramienta interactiva:* `herramientas/html/T2_Aliasing.html`.

3 El teorema del muestreo

La sección anterior estudió sinusoides. Para una señal cualquiera hace falta ver qué le ocurre a su espectro al muestrear.

3.1 Modelo con un tren de impulsos

La secuencia $x[n]$ no tiene transformada de Fourier continua, así que se construye una señal continua que contenga la misma información: el producto de $x(t)$ por un tren de impulsos (figura 6),

$$x_s(t) = x(t)\,p(t) = \sum_{n=-\infty}^{\infty} x(nT_s)\,\delta(t - nT_s), \qquad p(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s).$$

Por la propiedad $x(t)\delta(t - t_0) = x(t_0)\delta(t - t_0)$, cada impulso queda ponderado por una muestra.

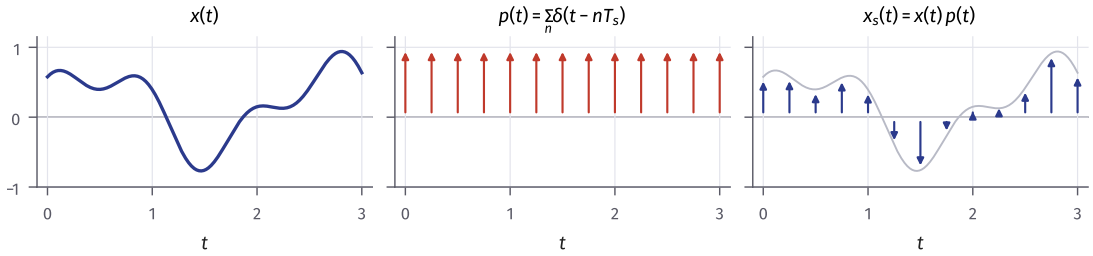


Figura 6. Muestreo ideal como producto por un tren de impulsos.

El tren $p(t)$ es periódico de periodo T_s, con coeficientes de Fourier

$$c_k = \frac{1}{T_s}\int_{-T_s/2}^{T_s/2} \delta(t)\,\mathrm{e}^{-jk\omega_s t}\,\mathrm{d}t = \frac{1}{T_s} \quad \text{para todo } k,$$

y una señal periódica tiene transformada $\sum_k 2\pi c_k \delta(\omega - k\omega_s)$. Por tanto:

Propiedad 3.1: Transformada del tren de impulsos

$$\sum_{n=-\infty}^{\infty} \delta(t - nT_s) \overset{\mathscr{F}}{\longleftrightarrow} \omega_s \sum_{k=-\infty}^{\infty} \delta(\omega - k\omega_s).$$

Impulsos separados T_s en el tiempo se transforman en impulsos separados $\omega_s = 2\pi/T_s$ en frecuencia.

3.2 Espectro de la señal muestreada

Un producto en el tiempo es una convolución en frecuencia dividida por 2π, y convolucionar con $\delta(\omega - k\omega_s)$ desplaza el espectro a $k\omega_s$:

$$\boxed{X_s(j\omega) = \frac{1}{2\pi}\,X(j\omega) * P(j\omega) = \frac{1}{T_s}\sum_{k=-\infty}^{\infty} X\big(j(\omega - k\omega_s)\big)} \tag{2}$$

El espectro de la señal muestreada es el de la original, escalado por $1/T_s$ y *replicado* cada ω_s (figura 7). Si las réplicas se solapan, se suman, y en la zona de solape ya no se puede saber qué parte venía de cada una: es el aliasing visto en frecuencia.

Definición 3.2: Teorema del muestreo (Nyquist–Shannon)

Sea $x(t)$ de *banda limitada*, $X(j\omega) = 0$ para $|\omega| > \omega_m$. Si

$$\omega_s > 2\,\omega_m \qquad (f_s > 2f_m),$$

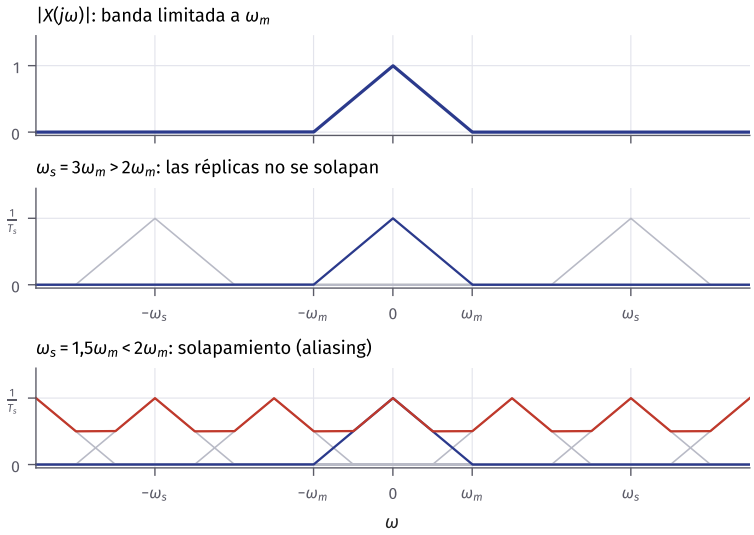


Figura 7. Espectro de la señal muestreada con muestreo suficiente (arriba) e insuficiente (abajo).

> $x(t)$ queda determinada de forma única por sus muestras $x(nT_s)$.

La demostración se lee en la figura 7: la réplica k ocupa $[k\omega_s - \omega_m, k\omega_s + \omega_m]$, y la central y la primera no se solapan si $\omega_m < \omega_s - \omega_m$. Entonces la réplica central es $X(j\omega)/T_s$ y un filtro paso bajo la aísla (apartado 4). La frecuencia $2f_m$ se llama *tasa de Nyquist*.

La desigualdad tiene que ser estricta: $\sin(\omega_m t)$ muestreada con $\omega_s = 2\omega_m$ da $\sin(\pi n) = 0$ en todas las muestras. Con una componente justo en $\omega_s/2$ las dos réplicas se tocan y la amplitud y la fase de esa componente quedan indeterminadas.

3.3 Visión continua y visión discreta

Transformando $x_s(t)$ término a término y comparando con la DTFT de $x[n]$,

$$X_s(j\omega) = \sum_n x(nT_s)\, \mathrm{e}^{-j\omega T_s n}, \qquad X(\mathrm{e}^{j\Omega}) = \sum_n x[n]\, \mathrm{e}^{-j\Omega n},$$

se ve que son la misma función con el cambio $\Omega = \omega T_s$:

$$X(\mathrm{e}^{j\Omega}) = X_s(j\omega)\big|_{\omega=\Omega/T_s} = \frac{1}{T_s} \sum_{k=-\infty}^{\infty} X\Big(j\,\frac{\Omega - 2\pi k}{T_s}\Big). \tag{3}$$

Esta relación tiene tres consecuencias que se usan continuamente:

- La banda $|\omega| \le \omega_s/2$ se transforma en $|\Omega| \le \pi$. Toda la información de la señal muestreada está en un intervalo de anchura 2π.
- La periodicidad 2π de la DTFT, que en el Tema 1 apareció como una propiedad matemática, *es* la replicación del espectro que produce el muestreo.
- Si $\omega_s = 2\omega_m$, la señal ocupa todo $[-\pi, \pi]$. Con *sobremuestreo*, por ejemplo $\omega_s = 4\omega_m$, ocupa $|\Omega| \le \pi/2$ y queda una banda libre que facilita los filtros analógicos (apartado 6.5).

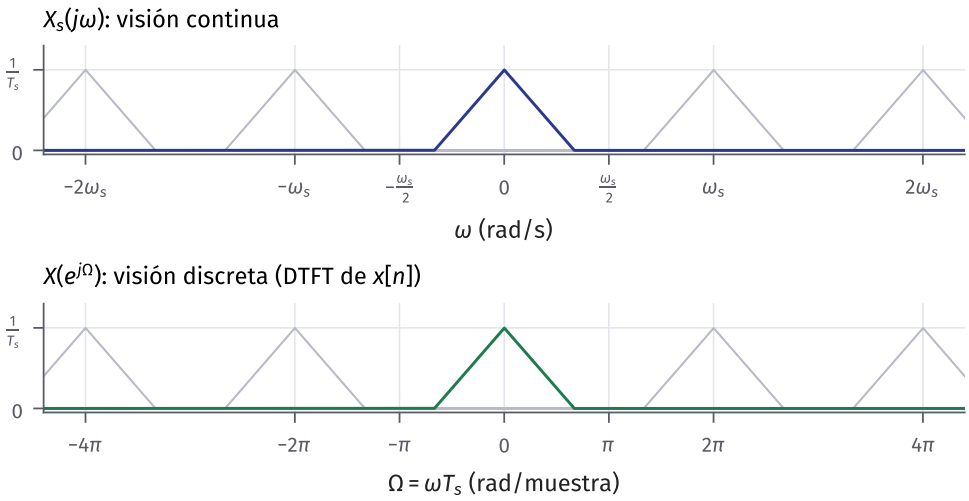


Figura 8. El mismo espectro en función de ω (rad/s) y de Ω (rad/muestra).

En código

Se muestrea a $f_s = 3\,\text{Hz}$ una señal sinc^2 de banda $f_m = 1\,\text{Hz}$ y se compara $T_s|X(e^{j\Omega})|$, dibujada frente a $\omega = \Omega/T_s$, con $|X(j\omega)|$.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

fm = 1.0                          # banda (Hz)
fs = 3.0                          # fs > 2 fm
Ts = 1 / fs
n = np.arange(-300, 301)
x = np.sinc(fm * n * Ts)**2       # banda fm
W, X = signal.freqz(x, 1, worN=2048,
                    whole=True)     # [0, π2)
W = W - np.pi                     # π[-, π)
X = np.fft.fftshift(X)
plt.plot(W / Ts, np.abs(X) * Ts)  # ω = Ω/Ts
plt.xlabel("ω (rad/s)")
plt.show()
```

Matlab

```matlab
fm = 1.0;                         % banda (Hz)
fs = 3.0;                         % fs > 2 fm
Ts = 1 / fs;
n = -300:300;
x = sinc(fm * n * Ts).^2;         % banda fm
[X, W] = freqz(x, 1, 2048, 'whole');
W = W - pi;                       % π[-, π)
X = fftshift(X);
plot(W / Ts, abs(X) * Ts)         % ω = Ω/Ts
xlabel('\omega (rad/s)')
```

3.4 Filtro antialiasing

Las señales reales no son de banda limitada. Una señal de duración finita tiene espectro infinito, y el ruido térmico ocupa todas las frecuencias. Lo que esté por encima de $f_s/2$ se plegará sobre la banda útil y, una vez muestreado, es imposible separarlo. Por eso, antes del conversor A/D se coloca un filtro analógico paso bajo, el *filtro antialiasing*, que atenúa todo lo que hay por encima de $f_s/2$ (figura 9).

Un filtro real no corta en seco: necesita una banda de transición. Por eso se muestrea con margen sobre la tasa de Nyquist. El audio de calidad CD ocupa hasta 20 kHz y se muestrea a 44,1 kHz, lo que deja una transición de 20 a 22,05 kHz; la telefonía limita la voz a 3,4 kHz y muestrea a 8 kHz.

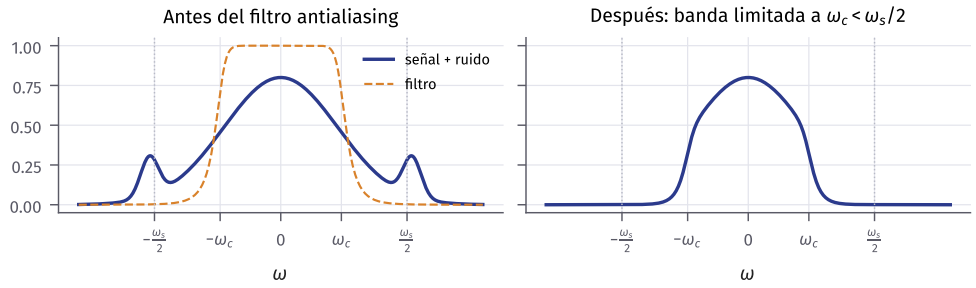


Figura 9. El filtro antialiasing elimina la parte del espectro que se plegaría sobre la banda útil.

3.5 Muestreo no ideal

Un circuito no puede generar impulsos ideales; se muestrea con pulsos de duración τ (figura 10). Hay dos variantes:

- **Muestreo natural**: $x_n(t) = x(t)\, p_\tau(t)$, donde p_τ es un tren de pulsos rectangulares de anchura τ. Sus coeficientes de Fourier son $c_k = \frac{\tau}{T_s}\,\mathrm{sinc}\left(\frac{k\tau}{T_s}\right)$, así que

$$X_n(j\omega) = \sum_k c_k\, X\big(j(\omega - k\omega_s)\big).$$

  Las réplicas no se deforman: solo se ponderan por c_k, y la central sigue siendo proporcional a $X(j\omega)$.

- **Muestreo de techo plano**: cada pulso mantiene el valor de la muestra, como hace un circuito de muestreo y retención (*sample and hold*). Es $x_s(t)$ convolucionada con un pulso de anchura τ, así que

$$X_{tp}(j\omega) = X_s(j\omega) \cdot \tau\ \mathrm{sinc}\left(\frac{\omega\tau}{2\pi}\right) e^{-j\omega\tau/2}.$$

  Ahora todas las réplicas, incluida la central, se multiplican por un sinc: es el *efecto de apertura*, que se corrige con un ecualizador o se hace despreciable con $\tau \ll T_s$. Con $\tau = T_s$ es exactamente el retenedor de orden cero de la apartado 4.4.

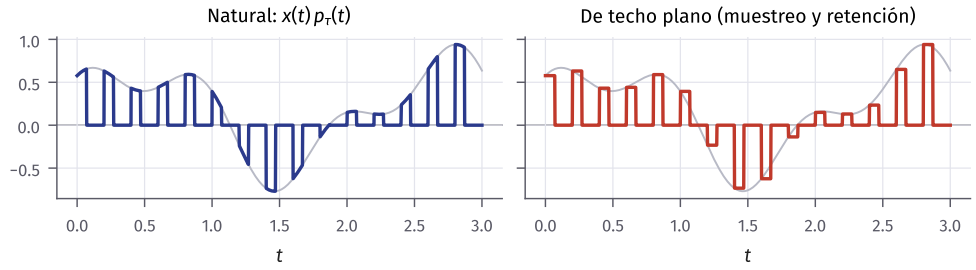


Figura 10. Muestreo natural y de techo plano.

Ejercicios 3–5. *Herramienta interactiva:* `herramientas/html/T2_Muestreo periódico.html` y `T2_Muestreo natural.html`.

4 Reconstrucción

Reconstruir es obtener una señal continua $y(t)$ a partir de la secuencia $x[n]$ y del periodo T_s. En general no tiene solución única, porque infinitas señales tienen las mismas muestras. Si $x(t)$ es de banda limitada y se cumple el teorema del muestreo, la réplica central de $X_s(j\omega)$ es $X(j\omega)/T_s$ y basta un filtro que se quede con ella.

4.1 Reconstrucción ideal

Definición 4.1: Filtro de reconstrucción ideal

$$H(j\omega) = T_s\,\Pi\!\left(\frac{\omega}{2\omega_c}\right) = \begin{cases} T_s & |\omega| \le \omega_c \\ 0 & |\omega| > \omega_c \end{cases}, \qquad \omega_m < \omega_c < \omega_s - \omega_m.$$

La ganancia T_s compensa el factor $1/T_s$ de (2), y la frecuencia de corte puede ser cualquiera entre el final de la réplica central y el principio de la siguiente (figura 11). La elección habitual es $\omega_c = \omega_s/2$, la única posible cuando el margen es mínimo.

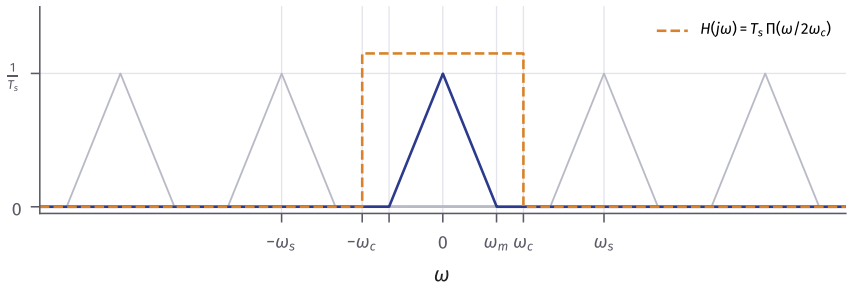


Figura 11. El filtro ideal de reconstrucción se queda con la réplica central.

Su respuesta al impulso es

$$h(t) = \frac{1}{2\pi}\int_{-\omega_c}^{\omega_c} T_s\, e^{j\omega t}\, d\omega = \frac{T_s}{\pi t}\sin(\omega_c t) = \frac{\omega_c T_s}{\pi}\,\mathrm{Sa}(\omega_c t),$$

con $\mathrm{Sa}(x) = \sin x/x$. Con $\omega_c = \omega_s/2 = \pi/T_s$ queda $h(t) = \mathrm{sinc}(t/T_s)$, con $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$: vale 1 en $t = 0$ y se anula en todos los demás múltiplos de T_s.

La salida del filtro es la convolución de $x_s(t)$ con $h(t)$, es decir, una suma de copias de h centradas en cada muestra:

Propiedad 4.2: Fórmula de interpolación de Whittaker–Shannon

Si $x(t)$ es de banda limitada a $\omega_m < \omega_s/2$,

$$x(t) = \sum_{n=-\infty}^{\infty} x[n]\ \mathrm{sinc}\left(\frac{t - nT_s}{T_s}\right).$$

En $t = nT_s$ solo contribuye la muestra n, porque las demás funciones sinc se anulan ahí; entre muestras contribuyen todas, ponderadas según su distancia (figura 12).

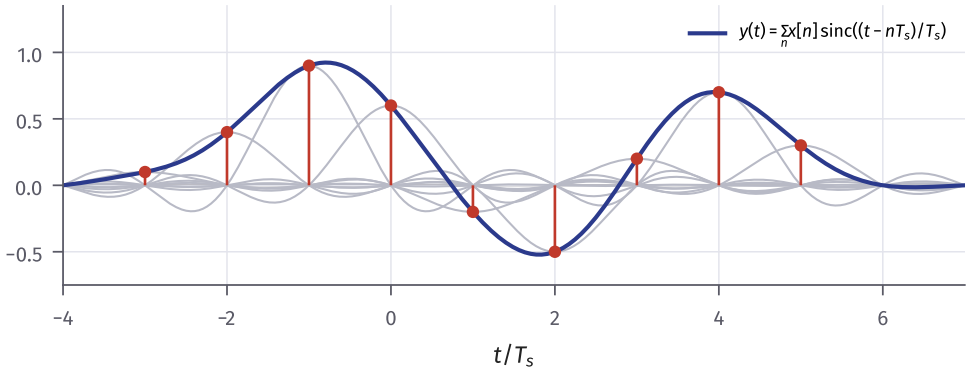
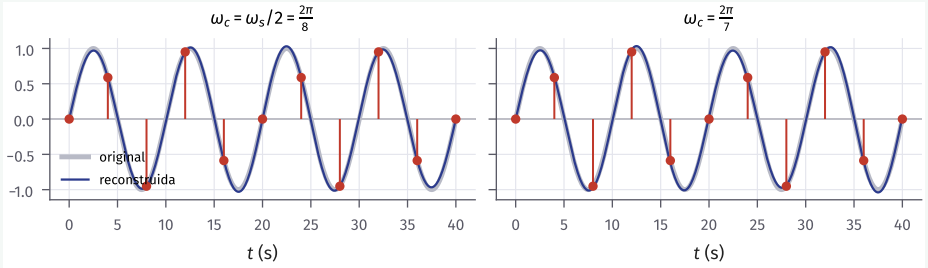


Figura 12. Interpolación con funciones sinc: la suma de las copias reproduce la señal.

Ejemplo 4.3: Elección de la frecuencia de corte

Se quiere reconstruir $x(t) = \sin(\omega_0 t)$, con $\omega_0 = 2\pi/10$ rad/s, a partir de sus muestras con $T_s = 4$ s. Como $\omega_s = 2\pi/4 = 5\pi/10 > 2\omega_0 = 4\pi/10$, se cumple el teorema. Las réplicas de la sinusoide están en $\pm\omega_0 + k\omega_s$; la más próxima a la central por arriba es $\omega_s - \omega_0 = 3\pi/10$. Cualquier ω_c con $2\pi/10 < \omega_c < 3\pi/10$ reconstruye exactamente la señal:

$$y(t) = \sum_n x[n]\,\frac{\omega_c T_s}{\pi}\,\mathrm{Sa}\left(\omega_c(t - nT_s)\right).$$



4.2 Por qué la reconstrucción ideal no es realizable

El filtro ideal tiene tres problemas prácticos:

- Es *no causal*: $h(t) \neq 0$ para $t < 0$, así que $y(t)$ depende de muestras futuras.
- Tiene *duración infinita*, y sinc decae solo como $1/t$: truncar la suma deja un error apreciable.
- Supone una señal de banda limitada, y ninguna señal de duración finita lo es.

Además, ningún filtro analógico tiene un corte abrupto. Fuera de línea (al remuestrear audio o escalar imágenes) se usa la fórmula truncada o con ventana. En tiempo real se usan *retenedores*, que interpolan con pocas muestras, seguidos de un filtro analógico de suavizado.

En código

Reconstrucción de un coseno de 2 Hz muestreado con $T_s = 0{,}1$ s con 101 muestras. El error no es exactamente cero porque la suma está truncada.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

Ts = 0.1                          # s
n = np.arange(-50, 51)
xn = np.cos(2*np.pi*2*n*Ts)       # 2 Hz < fs/2
t = np.linspace(-1, 1, 1001)
# y(t) = Σ x[n] sinc((t - nTs)/Ts)
S = np.sinc((t[:, None] - n*Ts) / Ts)
y = S @ xn
print(np.max(np.abs(y - np.cos(4*np.pi*t))))
plt.plot(t, y); plt.stem(n*Ts, xn)
plt.xlim(-1, 1); plt.show()
```

Matlab

```matlab
Ts = 0.1;                         % s
n = -50:50;
xn = cos(2*pi*2*n*Ts);            % 2 Hz < fs/2
t = linspace(-1, 1, 1001);
% y(t) = Σ x[n] sinc((t - nTs)/Ts)
S = sinc((t.' - n*Ts) / Ts);
y = S * xn.';
disp(max(abs(y.' - cos(4*pi*t))))
plot(t, y); hold on; stem(n*Ts, xn)
xlim([-1 1])
```

4.3 La reconstrucción como interpolación

Cualquier reconstrucción lineal e invariante con el desplazamiento de las muestras tiene la forma

$$y(t) = \sum_n x[n]\, h(t - nT_s) \quad \Longleftrightarrow \quad Y(j\omega) = H(j\omega)\, X_s(j\omega).$$

La *función de interpolación* $h(t)$ dice cuánto contribuye cada muestra según su distancia al instante t, y su transformada $H(j\omega)$ dice cuánto se deforma la réplica central y cuánto se atenúan las demás. El filtro ideal es el caso $h = \text{sinc}$; los retenedores usan funciones más cortas (figura 13).

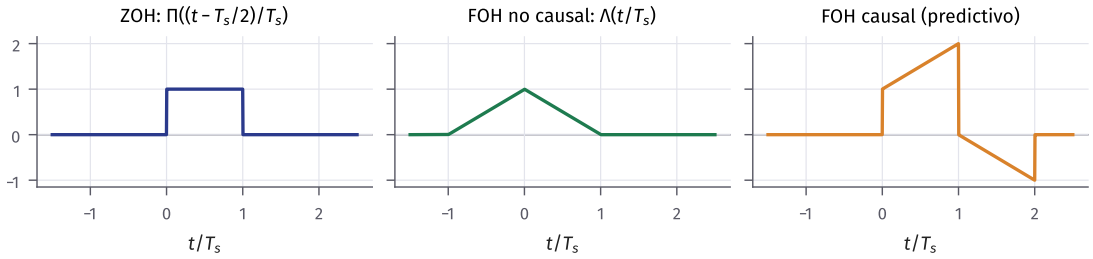


Figura 13. Funciones de interpolación de los retenedores de orden cero y uno.

4.4 Retenedor de orden cero

El retenedor de orden cero (ZOH, *zero-order hold*) mantiene cada muestra hasta la siguiente: $y(t) = x[n]$ para $nT_s \le t < (n+1)T_s$. Su función de interpolación es un pulso de anchura T_s que empieza en el origen:

$$h_0(t) = \Pi\!\left(\frac{t - T_s/2}{T_s}\right).$$

Su transformada es la de un pulso centrado, $T_s\,\text{sinc}(\omega/\omega_s)$, retrasada $T_s/2$:

$$\boxed{H_0(j\omega) = T_s\, e^{-j\omega T_s/2}\ \text{sinc}\left(\frac{\omega}{\omega_s}\right) = T_s\, e^{-j\omega T_s/2}\ \text{Sa}\left(\frac{\omega T_s}{2}\right)} \tag{4}$$

Errata frecuente

El retardo del ZOH es $e^{-j\omega T_s/2}$, con signo negativo: el pulso está a la *derecha* del origen. Con signo positivo el retenedor sería anticipativo.

El ZOH es causal y muy sencillo: es lo que hace un conversor D/A al mantener su salida hasta el siguiente código. Pero no es un buen filtro de reconstrucción (figura 14):

- **Atenúa la banda útil**. En $\omega_s/2$, $|H_0|/T_s = \operatorname{sinc}(1/2) = 2/\pi$, unos $-3{,}9$ dB. Para audio a 44,1 kHz, en 20 kHz la atenuación es de 3,2 dB.
- **No elimina las réplicas**, solo las atenúa como $1/\omega$. La primera imagen de un tono de 20 kHz, en 24,1 kHz, sale solo 4,8 dB por debajo.
- Introduce un **retardo** de media muestra.

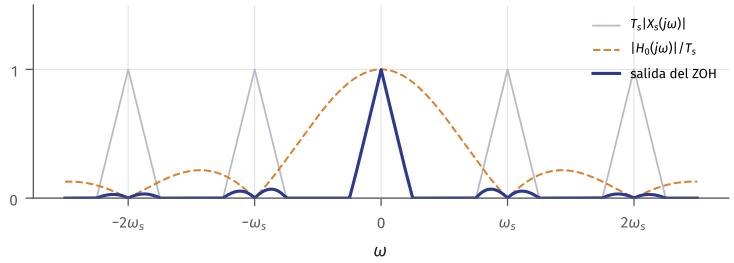


Figura 14. Módulo de la respuesta del ZOH frente al filtro ideal.

4.5 Retenedor de orden uno

El retenedor de orden uno (FOH, *first-order hold*) une las muestras con segmentos rectos. Hay dos versiones:

FOH no causal (interpolación lineal). Entre nT_s y $(n+1)T_s$,

$$y(t) = x[n] + \big(x[n+1] - x[n]\big)\frac{t - nT_s}{T_s}.$$

La función de interpolación es un triángulo de base $2T_s$ centrado en el origen, que es la convolución de dos pulsos de anchura T_s dividida por T_s:

$$h_1(t) = \Lambda\Big(\frac{t}{T_s}\Big) = \frac{1}{T_s}\,\Pi\Big(\frac{t}{T_s}\Big) * \Pi\Big(\frac{t}{T_s}\Big) \quad\Longrightarrow\quad \boxed{H_1(j\omega) = T_s\,\operatorname{sinc}^2\Big(\frac{\omega}{\omega_s}\Big)}$$

Es real y positiva (fase nula) y las réplicas caen como $1/\omega^2$. Usa la muestra siguiente, así que no es causal; retrasándolo una muestra se vuelve causal, con $H = T_s e^{-j\omega T_s}\operatorname{sinc}^2(\omega/\omega_s)$, a costa de un retardo T_s.

FOH causal (predictivo). Prolonga la recta que pasa por las dos últimas muestras:

$$y(t) = x[n] + \big(x[n] - x[n-1]\big)\frac{t - nT_s}{T_s}, \qquad nT_s \le t < (n+1)T_s.$$

La contribución de una muestra aislada es una rampa ascendente seguida de una descendente,

$$h(t) = \begin{cases} 1 + t/T_s & 0 \le t < T_s \\ 1 - t/T_s & T_s \le t < 2T_s \\ 0 & \text{en otro caso} \end{cases} \quad \Longrightarrow \quad H(j\omega) = T_s\,(1 + j\omega T_s)\,\mathrm{e}^{-j\omega T_s}\,\operatorname{sinc}^2\left(\frac{\omega}{\omega_s}\right).$$

El factor $1 + j\omega T_s$ amplifica las altas frecuencias: la salida tiene saltos en cada instante de muestreo y sobrepasa la señal cuando cambia la pendiente (figura 15).

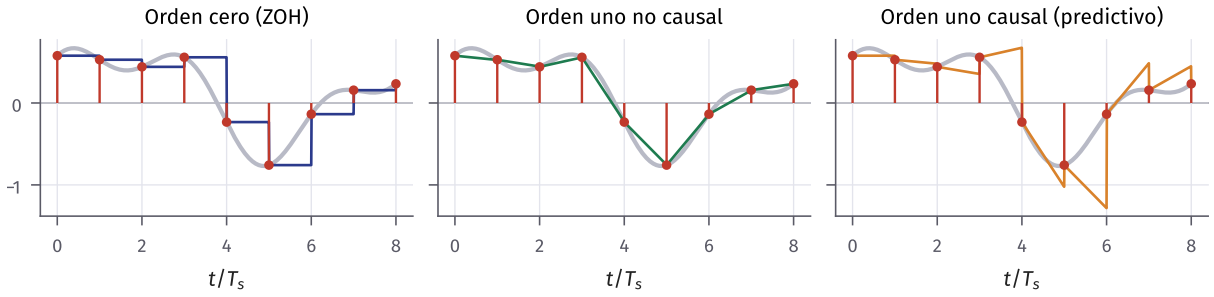


Figura 15. Los tres retenedores aplicados a la misma secuencia.

4.6 Comparación y conversión D/A

El cuadro 1 y la figura 16 resumen los retenedores. Ninguno reconstruye exactamente: deforman la banda útil y dejan pasar parte de las réplicas. Cuanto mayor es el orden, mejor atenúan las réplicas, pero necesitan más muestras y más retardo. Una cota útil del error para una señal suave: el ZOH comete un error de hasta $\max|x'|\,T_s$, y la interpolación lineal, de hasta $\max|x''|\,T_s^2/8$.

	$h(t)$	$\lvert H(j\omega)\rvert/T_s$	Causal
Ideal	$\operatorname{sinc}(t/T_s)$	$\Pi(\omega/\omega_s)$	No
ZOH	$\Pi\left(\frac{t-T_s/2}{T_s}\right)$	$\lvert\operatorname{sinc}(\omega/\omega_s)\rvert$	Sí
FOH no causal	$\Lambda(t/T_s)$	$\operatorname{sinc}^2(\omega/\omega_s)$	No (sí con retardo T_s)
FOH predictivo	dos rampas en $[0, 2T_s)$	$\lvert 1 + j\omega T_s\rvert\operatorname{sinc}^2(\omega/\omega_s)$	Sí

Cuadro 1. Comparación de los métodos de reconstrucción.

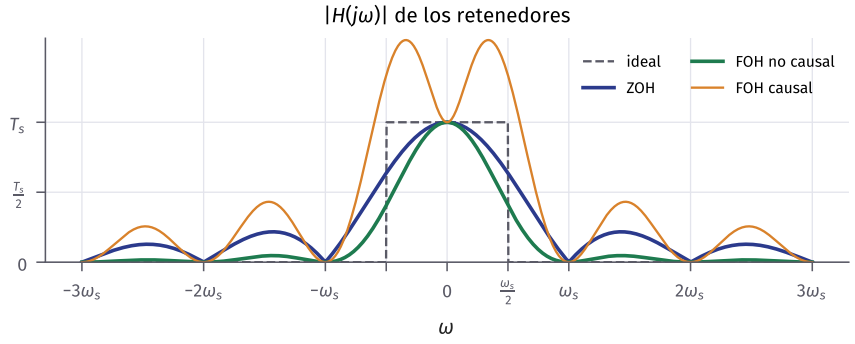


Figura 16. Respuesta en frecuencia de los retenedores.

Un conversor D/A real combina las piezas: una *compensación* digital con respuesta $1/\operatorname{sinc}(\omega/\omega_s)$ en la banda útil corrige la caída del ZOH; el conversor mantiene cada valor (ZOH); y un *filtro de suavizado* analógico elimina las réplicas que el ZOH deja pasar. Con sobremuestreo (apartado 6.5) las réplicas se alejan y ese filtro puede ser muy sencillo.

En código

Reconstrucción de una sinusoide con diez muestras por periodo con ZOH y FOH no causal. El error máximo del ZOH es 0,59 y el del FOH, 0,05.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

Ts = 0.1                          # s
n = np.arange(0, 11)
xn = np.sin(2*np.pi*1*n*Ts)       # 1 Hz
t = np.linspace(0, 1, 1001)
# Orden cero: se mantiene la última muestra
k = np.floor(t / Ts).astype(int)
y0 = xn[np.minimum(k, n[-1])]
# Orden uno (no causal): une las muestras
y1 = np.interp(t, n*Ts, xn)
x = np.sin(2*np.pi*t)
print(np.max(abs(y0 - x)), np.max(abs(y1 - x))
    )
plt.plot(t, y0, t, y1, t, x); plt.show()
```

Matlab

```matlab
Ts = 0.1;                         % s
n = 0:10;
xn = sin(2*pi*1*n*Ts);            % 1 Hz
t = linspace(0, 1, 1001);
% Orden cero: se mantiene la última muestra
y0 = interp1(n*Ts, xn, t, 'previous');
% Orden uno (no causal): une las muestras
y1 = interp1(n*Ts, xn, t, 'linear');
x = sin(2*pi*t);
disp([max(abs(y0 - x)), max(abs(y1 - x))])
plot(t, y0, t, y1, t, x)
```

Ejercicios 7–10. *Herramienta interactiva:* `herramientas/html/T2_Reconstrucción.html` y `T2_Retenedores.html`.

5 Cuantificación y codificación

El muestreo discretiza el tiempo, pero la amplitud de $x[n]$ sigue siendo *continua*. Un procesador solo puede almacenar números con un número finito de bits, así que cada muestra se aproxima por uno de un conjunto finito de valores. A diferencia del muestreo, esta aproximación siempre pierde información.

5.1 Cuantificador uniforme

Definición 5.1: Cuantificación

La *cuantificación* asigna a cada muestra $x[n]$ uno de L valores permitidos, los *niveles de cuantificación*:

$$x_q[n] = Q\big[x[n]\big].$$

En un cuantificador *uniforme* los niveles están separados por un *escalón* o *resolución* constante Δ. La diferencia $e[n] = x_q[n] - x[n]$ es el *error de cuantificación*.

Con b bits hay $L = 2^b$ niveles. Si la entrada está en el margen $[-V, V)$, de *fondo de escala* $FS = 2V$,

$$\Delta = \frac{FS}{2^b} = \frac{2V}{2^b}. \tag{5}$$

Es el convenio de los conversores A/D, donde Δ es el valor del bit menos significativo (1 LSB). En el margen bipolar los niveles son $k\Delta$ con $k = -2^{b-1}, \dots, 2^{b-1} - 1$: hay un nivel en cero y el margen es ligeramente asimétrico.

Otro convenio

Algunos textos colocan el primer y el último nivel exactamente en los extremos del intervalo $[x_{\min}, x_{\max}]$, y entonces $\Delta = \frac{x_{\max} - x_{\min}}{L-1}$. Las dos definiciones difieren en un nivel y dan las mismas conclusiones para L grande. En este curso se usa (5).

Si la entrada se sale del margen, el cuantificador *satura*: devuelve el nivel extremo y el error deja de estar acotado por Δ. Por eso el margen del conversor debe ajustarse a la señal, sin quedarse corto (saturación) ni largo (se desperdician niveles).

5.2 Redondeo y truncamiento

Hay dos formas habituales de elegir el nivel (figura 17):

- **Redondeo**: el nivel más próximo, $Q[x] = \Delta\ \mathrm{round}(x/\Delta)$. El error está en $-\Delta/2 \le e < \Delta/2$ y tiene media nula.
- **Truncamiento**: el nivel inmediatamente inferior, $Q[x] = \Delta\lfloor x/\Delta \rfloor$. El error está en $-\Delta < e \le 0$: tiene un sesgo de $-\Delta/2$.

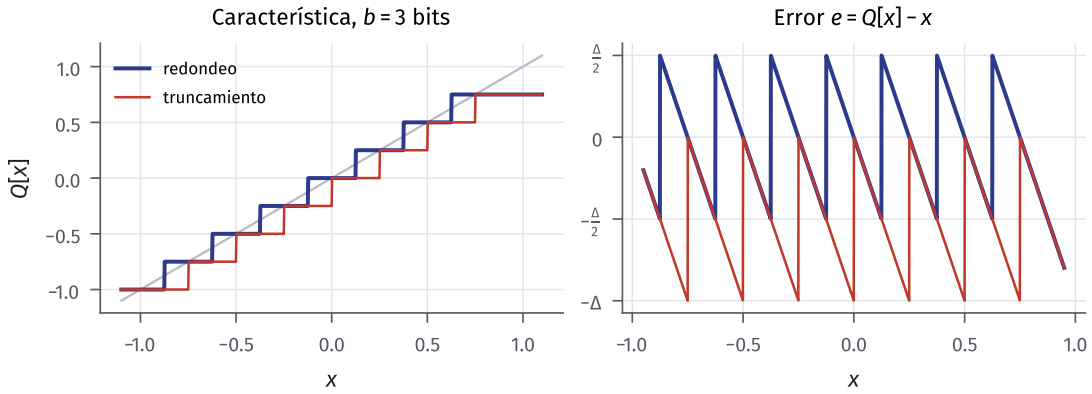


Figura 17. Característica de un cuantificador de 3 bits y error de cuantificación.

Ejemplo 5.2: Cuantificar $x[n] = 0{,}9^n$

Para representar exactamente los diez primeros valores ($n = 0, \dots, 9$) hace falta $\Delta = 10^{-9}$, porque $0{,}9^9 = 0{,}387420489$ tiene nueve decimales. Con un decimal, $\Delta = 0{,}1$:

		$x_q[n]$		$e[n]$	
n	$x[n]$	truncado	redondeado	truncado	redondeado
0	1	1,0	1,0	0	0
1	0,9	0,9	0,9	0	0
2	0,81	0,8	0,8	−0,01	−0,01
3	0,729	0,7	0,7	−0,029	−0,029
4	0,6561	0,6	0,7	−0,0561	0,0439
5	0,59049	0,5	0,6	−0,09049	0,00951
6	0,531441	0,5	0,5	−0,031441	−0,031441
7	0,4782969	0,4	0,5	−0,0782969	0,0217031
8	0,43046721	0,4	0,4	−0,03046721	−0,03046721
9	0,387420489	0,3	0,4	−0,087420489	0,012579511

Truncando, todos los errores son negativos y de hasta −0,09; redondeando, cambian de signo y no superan 0,05 en valor absoluto.

5.3 Ruido de cuantificación

El error de cuantificación es una función determinista de la señal, pero si la señal es activa (cruza muchos niveles entre muestra y muestra) y no satura, se comporta como un ruido aditivo (figura 18):

$$x_q[n] = x[n] + e[n],$$

con $e[n]$ uniformemente distribuido en $[-\Delta/2, \Delta/2)$, incorrelado con $x[n]$ y blanco (sus muestras son independientes entre sí). Su media es nula y su potencia es

$$\boxed{\sigma_e^2 = \frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e^2 \, \mathrm{d}e = \frac{\Delta^2}{12}} \tag{6}$$

Con truncamiento la varianza es la misma, pero la media es $-\Delta/2$. El modelo falla si la señal es lenta o pequeña frente a Δ: entonces el error se parece a la propia señal y aparece como distorsión armónica. Las herramientas para describir señales aleatorias se estudian en el Tema 6.

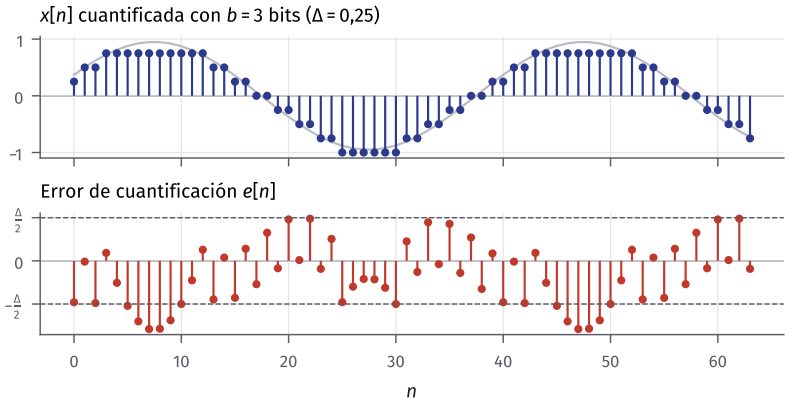


Figura 18. Señal cuantificada con 3 bits y su error de cuantificación.

La calidad de la cuantificación se mide con la *relación señal a ruido de cuantificación*,

$$\mathrm{SQNR} = 10\log_{10}\frac{P_x}{\sigma_e^2}\ \mathrm{dB}.$$

Para una sinusoide que ocupa todo el margen, de amplitud V, $P_x = V^2/2$, y con (5) y (6):

$$\boxed{\mathrm{SQNR} = 10\log_{10}\frac{V^2/2}{(2V)^2/(12\cdot 2^{2b})} = 10\log_{10}\left(\frac{3}{2}\,2^{2b}\right) \approx 6{,}02\,b + 1{,}76\ \mathrm{dB}} \tag{7}$$

Cada bit añade unos 6 dB (figura 19). Un CD, con 16 bits, alcanza unos 98 dB. La fórmula supone que la señal aprovecha todo el margen: si su amplitud es $V/10$, la SQNR baja 20 dB, lo que equivale a perder más de tres bits.

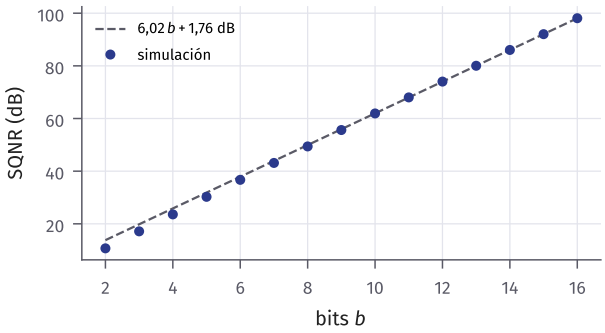


Figura 19. SQNR medida y teórica en función del número de bits.

En código

Cuantificación por redondeo con 8 bits de una sinusoide de amplitud $0{,}99\,V$. La varianza del error coincide con $\Delta^2/12$ y la SQNR medida, 49,7 dB, con los 49,9 dB de (7).

Python

```python
import numpy as np

b = 8                          # bits
V = 1.0                        # margen [-V, V)
D = 2*V / 2**b                 # escalón Δ
n = np.arange(100_000)
x = 0.99*V*np.sin(2*np.pi*0.01234*n)
xq = D * np.round(x / D)       # redondeo
e = xq - x
print(np.var(e), D**2/12)      # ≈ Δ²/12
snr = 10*np.log10(np.var(x) / np.var(e))
print(snr, 6.02*b + 1.76)      # dB
```

Matlab

```matlab
b = 8;                         % bits
V = 1.0;                       % margen [-V, V)
D = 2*V / 2^b;                 % escalón Δ
n = 0:99999;
x = 0.99*V*sin(2*pi*0.01234*n);
xq = D * round(x / D);         % redondeo
e = xq - x;
disp([var(e), D^2/12])         % ≈ Δ²/12
snr = 10*log10(var(x) / var(e));
disp([snr, 6.02*b + 1.76])     % dB
```

5.4 Codificación

La codificación asigna a cada nivel una palabra binaria. Con b bits hay 2^b palabras, así que para L niveles hace falta

$$2^b \geq L \quad\Longrightarrow\quad b \geq \log_2 L.$$

Por ejemplo, 6 niveles necesitan $b \geq \log_2 6 \approx 2{,}585$, es decir, 3 bits (y sobran dos palabras).

Para un margen bipolar, con niveles $k\Delta$, se usan sobre todo dos códigos (cuadro 2): el *binario desplazado*, que codifica $k + 2^{b-1}$ en binario natural y es el que entregan muchos conversores, y el *complemento a dos*, que es el de la aritmética entera de los procesadores. Pasar de uno a otro es tan sencillo como invertir el bit más significativo.

k	$x_q = k\Delta$	binario desplazado	complemento a dos
3	3Δ	111	011
2	2Δ	110	010
1	Δ	101	001
0	0	100	000
-1	$-\Delta$	011	111
-2	-2Δ	010	110
-3	-3Δ	001	101
-4	-4Δ	000	100

Cuadro 2. Códigos de un cuantificador bipolar de 3 bits.

5.5 Parámetros de un conversor A/D

Un conversor A/D se describe por:

- Su **resolución** b y su **tensión de referencia** V_{ref}. Con un margen unipolar $[0, V_{\text{ref}})$, $1\,\text{LSB} = \Delta = V_{\text{ref}}/2^b$.
- Su **rango dinámico**: el cociente entre la mayor y la menor amplitud que distingue,

$$\text{DR} = 20\log_{10}\frac{V_{\text{ref}}}{\Delta} = 20\log_{10} 2^b \approx 6{,}02\,b\ \text{dB}.$$

  Ojo: «rango dinámico» designa a veces el intervalo de valores de la entrada. Aquí lo reservamos para el cociente en decibelios y llamamos *margen* al intervalo.
- Su **frecuencia de muestreo** máxima, ligada al tiempo de conversión.
- Su **arquitectura**: *flash* (muy rápida, pocos bits), de aproximaciones sucesivas (SAR, el compromiso habitual en microcontroladores) o sigma-delta (muchos bits gracias al sobremuestreo, apartado 6.5).

Ejemplo 5.3: Conversor de un microcontrolador

Un conversor de 10 bits con $V_{\text{ref}} = 3{,}3\,\text{V}$ tiene $\Delta = 3{,}3/1024 \approx 3{,}22\,\text{mV}$ y $\text{DR} \approx 60{,}2\,\text{dB}$. Para una sinusoide que ocupe todo el margen, la SQNR es de unos 62 dB. Si se quiere distinguir variaciones de 0,1 mV, hacen falta $b \geq \log_2(3{,}3/10^{-4}) \approx 15{,}01$, es decir, 16 bits.

Ejercicios 11–13.

6 Muestreo de señales discretas

Muchas veces hay que cambiar la frecuencia de muestreo de una señal que ya es discreta: pasar audio de 48 kHz a los 8 kHz de la telefonía, o de los 44,1 kHz del CD a los 48 kHz del vídeo; cambiar el tamaño de una imagen; o procesar a la menor frecuencia posible para ahorrar cálculo. Volver al dominio analógico y muestrear de nuevo añadiría ruido y coste. Todo lo visto para señales continuas tiene su versión discreta: réplicas, aliasing y filtros (figura 20).

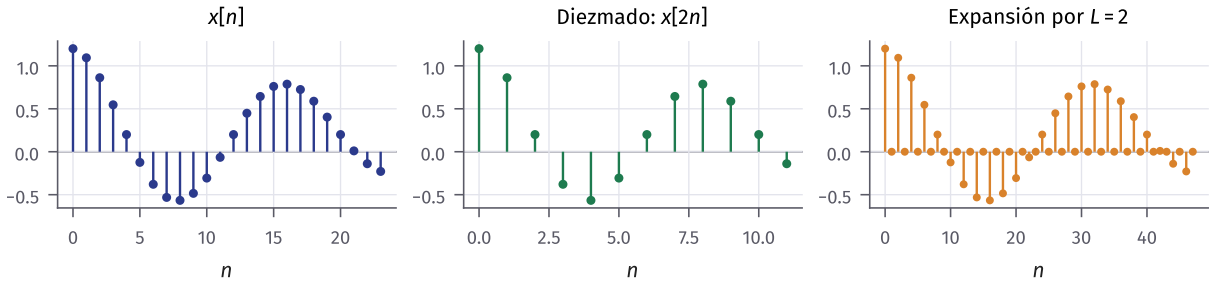


Figura 20. Una secuencia, su versión diezmada por 2 y su versión expandida por 2.

6.1 Muestreo de una secuencia

Muestrear una secuencia con periodo N es conservar una de cada N muestras y poner a cero las demás:

$$x_p[n] = x[n]\,p[n], \qquad p[n] = \sum_{k=-\infty}^{\infty} \delta[n - kN].$$

El tren $p[n]$ es periódico de periodo N, con desarrollo en serie de Fourier discreta

$$p[n] = \frac{1}{N} \sum_{k=0}^{N-1} \mathrm{e}^{j2\pi kn/N}.$$

Multiplicar por $\mathrm{e}^{j2\pi kn/N}$ desplaza el espectro $2\pi k/N$, así que

Propiedad 6.1: Espectro de una secuencia muestreada

$$X_p(\mathrm{e}^{j\Omega}) = \frac{1}{N} \sum_{k=0}^{N-1} X\big(\mathrm{e}^{j(\Omega - 2\pi k/N)}\big).$$

Es la versión discreta de (2): aparecen $N-1$ réplicas separadas $\Omega_s = 2\pi/N$. Si $x[n]$ tiene banda limitada a Ω_m, no se solapan cuando $\Omega_m < \pi/N$, y entonces $x[n]$ se recupera de $x_p[n]$ con un filtro paso bajo ideal de ganancia N y corte π/N.

6.2 Diezmado

Las muestras nulas de $x_p[n]$ no aportan nada; eliminarlas es *diezmar* (o submuestrear) por un factor M:

$$x_d[n] = x[nM] = x_p[nM].$$

La nueva frecuencia de muestreo es f_s/M. Como $X_d(\mathrm{e}^{j\Omega}) = \sum_n x_p[nM]\mathrm{e}^{-j\Omega n} = X_p(\mathrm{e}^{j\Omega/M})$, el espectro se *expande* por M:

Propiedad 6.2: Diezmado

$$X_d(\mathrm{e}^{j\Omega}) = \frac{1}{M} \sum_{k=0}^{M-1} X\big(\mathrm{e}^{j(\Omega - 2\pi k)/M}\big).$$

No hay aliasing si $\Omega_m < \pi/M$.

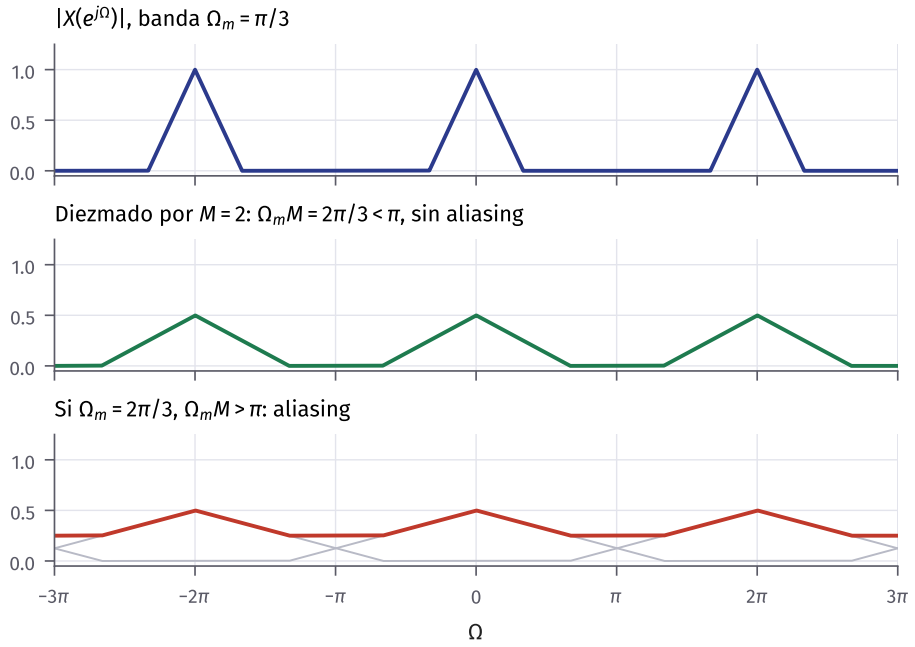


Figura 21. Diezmado por $M = 2$ sin aliasing (centro) y con aliasing (abajo).

El resultado sigue siendo periódico de periodo 2π, como debe ser la DTFT de cualquier secuencia: la réplica k queda centrada en $2\pi k$ (figura 21). Es lo esperable de (1): con la nueva frecuencia de muestreo, cada frecuencia en hercios corresponde a una frecuencia discreta M veces mayor.

Si $x[n]$ ocupa más de π/M, antes de diezmar hay que filtrarla con un paso bajo de ganancia 1 y corte π/M (figura 22). Es el mismo papel que el filtro antialiasing analógico: se pierde la banda alta, pero no se mezcla con la baja. El filtro trabaja a la frecuencia alta, aunque solo hace falta calcular una de cada M salidas.

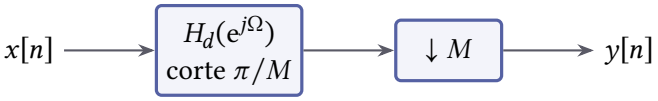


Figura 22. Diezmador con filtro antialiasing.

Ejemplo 6.3: De 48 kHz a 8 kHz

El factor es $M = 6$. El filtro corta en $\pi/6$, que a $f_s = 48\,\text{kHz}$ equivale a $\frac{\pi/6}{2\pi} \cdot 48 = 4\,\text{kHz}$: justo la frecuencia de Nyquist de la señal de salida.

6.3 Expansión e interpolación

Para aumentar la frecuencia de muestreo por un factor entero L, primero se insertan $L-1$ ceros entre cada par de muestras (*expansión*):

$$x_e[n] = \begin{cases} x[n/L] & n = 0, \pm L, \pm 2L, \dots \\ 0 & \text{en otro caso} \end{cases} \qquad \Longrightarrow \qquad X_e(e^{j\Omega}) = \sum_k x[k] e^{-j\Omega k L} = X(e^{j\Omega L}).$$

El espectro se *comprime* por L y en $[-\pi, \pi]$ aparecen $L-1$ copias adicionales, las *imágenes* (figura 23). Un *filtro de interpolación* paso bajo de ganancia L y corte π/L las elimina y «rellena» los ceros. La ganancia L compensa los ceros insertados: el valor medio de $x_e[n]$ es $1/L$ del de $x[n]$.

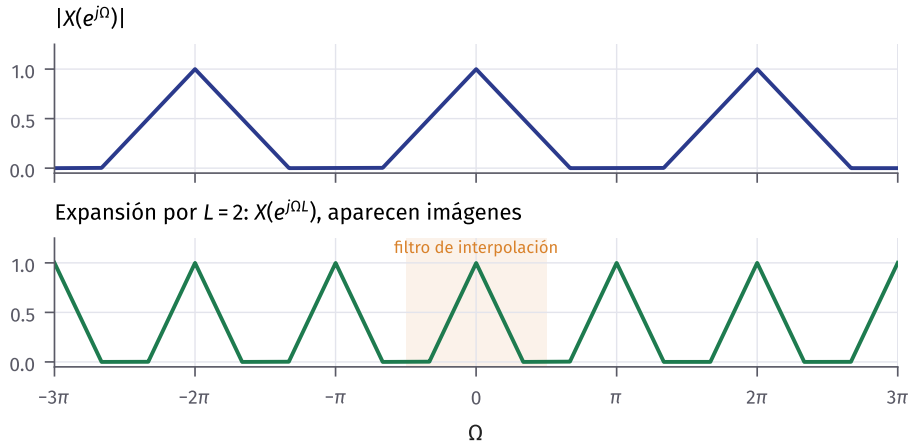


Figura 23. Expansión por $L = 2$: el espectro se comprime y aparecen imágenes.

Con el filtro ideal, cuya respuesta al impulso es $h[n] = \operatorname{sinc}(n/L)$, la salida es la interpolación de banda limitada de las muestras,

$$y[n] = \sum_k x[k]\ \operatorname{sinc}\left(\frac{n - kL}{L}\right),$$

que es la fórmula de Whittaker–Shannon evaluada en los nuevos instantes; en particular, $y[kL] = x[k]$. Con filtros más sencillos se obtienen los retenedores discretos: repetir cada muestra L veces ($h[n] = 1$ para $0 \le n < L$, orden cero) o interpolar linealmente ($h[n] = \Lambda(n/L)$, orden uno).

6.4 Cambio de frecuencia por un factor racional

Para cambiar la frecuencia por un factor racional L/M se interpola por L y después se diezma por M (figura 24). En ese orden no se pierde banda innecesariamente, y los dos filtros, el de interpolación y el antialiasing, se funden en uno solo con ganancia L y corte $\min(\pi/L, \pi/M)$.

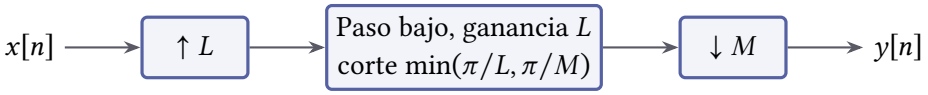


Figura 24. Cambio de frecuencia de muestreo por un factor L/M.

Ejemplo 6.4: De 44,1 kHz a 48 kHz

$48\,000/44\,100 = 160/147$, que ya es irreducible. Se interpola por $L = 160$, lo que da una frecuencia intermedia de 7,056 MHz, se filtra con corte $\pi/160$ y se diezma por $M = 147$. Calcular directamente así sería muy caro; en la práctica se usan estructuras *polifásicas*, que no calculan las muestras que se van a descartar ni multiplican por los ceros insertados.

En código

Diezmado por 4 con filtro antialiasing incluido (`decimate`) y cambio de 44,1 a 48 kHz con una estructura polifásica (`resample_poly` en Python, `resample` en Matlab).

Python

```python
import numpy as np
from scipy import signal

fs = 44_100                       # Hz (CD)
t = np.arange(0, 0.1, 1/fs)
x = np.sin(2*np.pi*1000*t)        # 1 kHz
# Diezmado por 4 con filtro antialiasing
x4 = signal.decimate(x, 4)        # fs/4
# Cambio racional 44,1 kHz -> 48 kHz
y = signal.resample_poly(x, 160, 147)
print(len(x), len(x4), len(y))
```

Matlab

```matlab
fs = 44100;                       % Hz (CD)
t = 0:1/fs:0.1-1/fs;
x = sin(2*pi*1000*t);             % 1 kHz
% Diezmado por 4 con filtro antialiasing
x4 = decimate(x, 4);              % fs/4
% Cambio racional 44,1 kHz -> 48 kHz
y = resample(x, 160, 147);
disp([length(x), length(x4), length(y)])
```

6.5 Conversores con sobremuestreo

Los conversores modernos muestrean mucho más deprisa de lo que exige el teorema y ajustan la frecuencia en digital:

- **A/D**: se muestrea a Kf_s y se diezma por K. El filtro antialiasing analógico puede ser muy suave, porque su transición va de f_m a $Kf_s - f_m$; el filtro abrupto lo hace el diezmador digital. Además, el ruido de cuantificación, de potencia $\Delta^2/12$, se reparte uniformemente en $[-\pi, \pi]$ y el filtro del diezmador solo deja pasar la fracción $1/K$:

  $$\mathrm{SQNR} \approx 6{,}02\,b + 1{,}76 + 10\log_{10} K \ \mathrm{dB}.$$

  Cada factor 4 de sobremuestreo gana unos 6 dB, es decir, un bit.

- **D/A**: se interpola por K antes del conversor. Las imágenes se alejan hasta Kf_s, el filtro de suavizado es sencillo y la caída del ZOH en la banda útil es casi nula.

Los conversores *sigma-delta* van más allá: un lazo de realimentación *conforma* el ruido de cuantificación para empujarlo fuera de la banda útil, de modo que con un cuantificador de un solo bit a varios megahercios se logran de 16 a 24 bits efectivos en la banda de audio.

Ejercicios 14–16.

Resumen del tema

- Muestreo periódico: $x[n] = x(nT_s)$; frecuencia discreta $\Omega = \omega T_s = 2\pi f/f_s$.
- Las frecuencias $\pm\omega_0 + k\omega_s$ dan las mismas muestras (aliasing). El alias fundamental es el único con $|\omega| \le \omega_s/2$.
- Espectro de la señal muestreada: $X_s(j\omega) = \frac{1}{T_s}\sum_k X(j(\omega - k\omega_s))$, y $X(e^{j\Omega}) = X_s(j\Omega/T_s)$.
- Teorema del muestreo: una señal de banda limitada a ω_m queda determinada por sus muestras si $\omega_s > 2\omega_m$. En la práctica, filtro antialiasing y margen.
- Reconstrucción ideal: filtro $T_s\Pi(\omega/2\omega_c)$ e interpolación con sinc. No es causal ni realizable.

- Retenedores: ZOH, $H_0 = T_s e^{-j\omega T_s/2}\,\mathrm{sinc}(\omega/\omega_s)$; FOH, $H_1 = T_s\,\mathrm{sinc}^2(\omega/\omega_s)$. Un D/A real usa ZOH, compensación y filtro de suavizado.
- Cuantificación uniforme: $\Delta = FS/2^b$; error uniforme de potencia $\Delta^2/12$; SQNR $\approx 6{,}02\,b + 1{,}76$ dB. Codificar L niveles requiere $b \geq \log_2 L$ bits.
- Diezmado: $X_d = \frac{1}{M}\sum_k X(e^{j(\Omega-2\pi k)/M})$, sin aliasing si $\Omega_m < \pi/M$. Expansión: $X_e = X(e^{j\Omega L})$, con imágenes que elimina un filtro de ganancia L y corte π/L. Cambio racional: interpolar, filtrar y diezmar.
- El sobremuestreo simplifica los filtros analógicos y mejora la SQNR en $10\log_{10} K$ dB.

Bibliografía

- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulo de aplicaciones de la transformada de Fourier, apartados de muestreo. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.
- A. V. Oppenheim y A. S. Willsky, *Señales y sistemas*, 2.ª ed. Prentice Hall, 1997. Capítulo 7, la referencia para el muestreo, la reconstrucción y el muestreo de señales discretas.
- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos 1 (conversión A/D y D/A), 6 (muestreo y reconstrucción) y 11 (cambio de frecuencia de muestreo).
- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulo 4, con el diezmado, la interpolación, el sobremuestreo y el ruido de cuantificación.